

Assignment-03 Report

[Actors-based]

Programmazione Concorrente e Distribuita [25-26]

Alex Mazzoni

`alex.mazzoni3@studio.unibo.it`

May 19, 2026

Contents

1	Exercise 1: Smart Home Alarm System (Scala - Pekko)	2
1.1	Analisi del Problema	2
1.1.1	Macchina a Stati e Vincoli Temporali	2
1.1.2	Gestione delle Zone e Tolleranza ai Guasti	2
1.2	Design e Implementazione	3
1.2.1	Gerarchia degli Attori e Routing	3
1.2.2	Modellazione del Guardian	4
1.2.3	Gestione delle Zone e dei Sensori	4
1.2.4	Supervisione e Signal	4
1.3	Conclusioni e Test	4
2	Exercise 2: Odds-and-Evens Game (Go)	5
2.1	Analisi del Problema	5
2.1.1	Struttura dei Canali	5
2.1.2	Sincronizzazione e Deadlock	5
2.2	Design e Implementazione	6
2.2.1	Routing Statico dei Canali	6
2.2.2	Sincronizzazione tra Giocatori	6
2.2.3	Svolgimento della battaglia	6
2.3	Conclusioni e Output	6

1 Exercise 1: Smart Home Alarm System (Scala - Pekko)

L'obiettivo dell'esercizio è la progettazione e implementazione di un sistema di allarme domestico. Il sistema deve gestire segnali asincroni provenienti da una rete di sensori, e interazioni utente tramite un tastierino, reagendo in modo differente in base al proprio stato.

1.1 Analisi del Problema

Inizialmente sono stati analizzati gli stati e le transizioni del sistema, identificando chiaramente le fasi di **Disarmed**, **Exit Delay**, **Armed**, **Entry Delay** e **Alarm**. Ogni stato ha comportamenti specifici in risposta a eventi come l'inserimento del PIN, il rilevamento di un sensore o lo scadere di un timer.

1.1.1 Macchina a Stati e Vincoli Temporal

Il cuore del sistema in grado di gestire i principali stati, è il Guardian, che deve implementare una macchina a stati finiti per garantire che le transizioni avvengano in modo coerente e prevedibile. Inoltre, i vincoli temporali (Exit Delay ed Entry Delay) richiedono l'uso di timer non bloccanti, che devono essere gestiti in modo asincrono senza compromettere la reattività del sistema.

1.1.2 Gestione delle Zone e Tolleranza ai Guasti

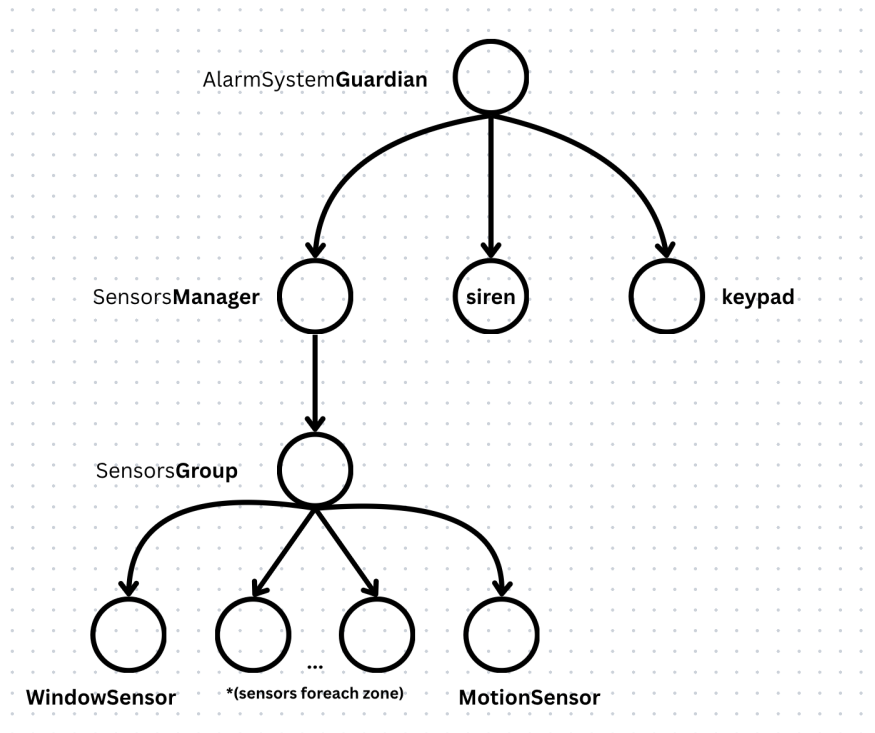
Il sistema richiede la gestione di sensori divisi in **Zone**, introducendo la necessità di un routing efficiente dei messaggi verso sotto-gruppi specifici di dispositivi. Inoltre, trattandosi di un sistema di sicurezza, l'architettura deve prevedere la possibilità che i sensori hardware si disconnettano o falliscano, richiedendo un meccanismo reattivo per notificare la centralina della manomissione.

1.2 Design e Implementazione

La soluzione sfrutta pienamente la natura gerarchica e reattiva degli attori in Pekko, suddividendo le responsabilità, promuovendo l'incapsulazione, e mantenendo lo stato isolato all'interno di ciascun attore.

1.2.1 Gerarchia degli Attori e Routing

L'architettura è guidata dall'attore `AlarmSystemGuardian`, che funge da centralina e coordinatore principale. Sotto di esso, il sistema si espande ad albero:



- **KeypadActor** (`keypad`): Si occupa di segnalare input dell'utente.
- **SirenActor** (`siren`): Si occupa di attivare l'allarme sonoro.
- **SensorsManager**: Un attore intermediario che funge da router. `AlarmSystemGuardian` delega a lui la totale creazione e gestione dei sensori.
- **SensorsGroup**: Mantiene la suddivisione dei sensori in zone, occupandosi di propagare i comandi di armamento / disarmamento in modalità broadcast ai **SensorsActors**.
- **SensorsActors**: Definisce il comportamento di ciascun sensore individuale (`MotionSensor` e `WindowSensor`).

1.2.2 Modellazione del Guardian

Invece di utilizzare variabili di stato mutabili condivise, la macchina a stati nel **Alarm-SystemGuardian** è stata implementata sfruttando il *Behavior Switching* di Pekko. Ogni stato del sistema restituisce un nuovo **Behavior[Command]** che definisce esattamente quali messaggi sono accettati e come reagire in quella specifica fase, ignorando o gestendo diversamente i trigger fuori contesto.

Per i vincoli temporali, è stato utilizzato il pattern **Behaviors.withTimers**. L'attore avvia timer non bloccanti (*startSingleTimer*) che, allo scadere, auto-inviano un messaggio interno (es. `ExitTimeExpired` o `EntryTimeExpired`), innescando la transizione di stato in modo puramente guidato dagli eventi.

1.2.3 Gestione delle Zone e dei Sensori

Il **SensorsManager** definisce con un factory pattern le regole di **spawn** dei sensori: Per ogni **Zones.Zone** vengono creati un **WindowSensor** e un **MotionSensor**, tale pattern è utilizzato per delegare al **SensorsGroup** la creazione dei sensori, per quella specifica zona, e mantenere così una chiara separazione delle responsabilità.

I sensori sono progettati in modo generico con **GenericSensor**, che definisce il comportamento comune di tutti i sensori, in questo modo è possibile creare più tipologie di sensori, che avranno lo stesso comportamento base nel sistema, ovvero l'invio del segnale di allerta (`Trigger` e `SetArmed(isArmed)`).

La logica di armamento per zona, è gestita dal Guardian, è lui che decide quali zone devono essere armate o disarmate, il sensore si occuperà di inviare il segnale di allerta, solo se il sensore è armato.

1.2.4 Supervisione e Signal

Per garantire la robustezza e rilevare i guasti ai sensori, è stato implementato un meccanismo di intercezione di segnali per i **SensorsActors**. Nel **SensorsGroup**, ogni sensore generato viene osservato esplicitamente tramite `context.watch(sensor)`. Se un sensore lancia un'eccezione o viene terminato, il **SensorsGroup** intercetta il segnale **Terminated** tramite la `receiveSignal` e notifica immediatamente il Guardian con un messaggio di `SensorOffline`, permettendo al sistema di gestire il caso in base al suo stato.

Per gli attori **KeypadActor** e **SirenActor**, il Guardian applica invece una `SupervisorStrategy.restart` per gestire eventuali fallimenti transitori e ripristinare il loro stato pulito.

1.3 Conclusioni e Test

Ho utilizzato scalatest per implementare non proprio dei test automatizzati, ma un ambiente hardcoded, rendendo intuitivi i test principali. Inoltre nel main è presente un'esecuzione interattiva, che permette di inserire I/O da console per testare il sistema in modo dinamico.

2 Exercise 2: Odds-and-Evens Game (Go)

L'obiettivo dell'esercizio è la progettazione di un gioco basato sul paradigma ad attori, in cui $N = 2^m$ giocatori si sfidano in un torneo a eliminazione diretta, composto da m round. Il vincolo principale è l'assenza di memoria condivisa, che impone una comunicazione esclusivamente tramite messaggi.

2.1 Analisi del Problema

Il design dell'approccio concorrente, è stato guidato dall'osservazione del torneo rappresentato come un albero binario completo, dove ogni nodo rappresenta una sfida tra due giocatori. I vincitori di ogni sfida avanzano al round successivo, fino a quando non rimane un solo vincitore finale.

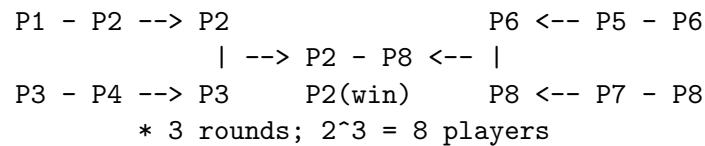


Figure 1: Rappresentazione ad albero del torneo

2.1.1 Struttura dei Canali

Analizzando lo schema di esempio in Figura 1 mostra come i giocatori in un ordine ben definito si sfidano in ogni round; I simboli '-' e '|' indicano una battaglia tra due giocatori (attori).

Definiamo così che il numero di canali necessari per round, che equivale al numero di battaglie per round r , dato da 2^{m-r} .

Con l'esempio rappresentato in Figura 1 con $m = 3$, al primo round $r = 1$, avremo 2^2 canali (P1 vs P2, P3 vs P4, P5 vs P6, P7 vs P8). E così via, fino al round finale m dove avremo un solo canale per la sfida finale tra i due vincitori dei round precedenti.

2.1.2 Sincronizzazione e Deadlock

Per ogni battaglia gli attori utilizzeranno un unico canale, tale approccio in un ambiente con messaggi sincroni, introduce un potenziale rischio di deadlock.

Se entrambi i giocatori coinvolti in una battaglia tentano di inviare un messaggio contemporaneamente, si bloccheranno reciprocamente, poiché entrambi attendono che l'altro riceva prima di poter procedere.

Per risolvere questo problema, è necessario implementare una strategia di sincronizzazione che garantisca che in ogni battaglia ci sia sempre un giocatore che assuma il ruolo di **Server** (che attenda la ricezione del messaggio) e un giocatore che assuma il ruolo di **Client** (che invii il primo messaggio).

In una rappresentazione ad albero binario (Figura 1), possiamo notare che i giocatori che si sfidano provengono sempre o da destra o da sinistra, questo ci permette di definire una regola semplice per assegnare i ruoli.

2.2 Design e Implementazione

La soluzione implementata in Go utilizza un modello **decentralizzato** (**Symmetric Peer-to-Peer**) per le sfide, supportato da una fase di setup deterministica iniziale.

2.2.1 Routing Statico dei Canali

Per eliminare la necessità di un coordinatore centrale che smisti i messaggi dinamicamente, è stata creata una matrice di canali pre-allocata. Nel Flusso del **main**, vengono definiti i canali di battaglia per ogni giocatore calcolati in base al proprio ID.

Ogni giocatore utilizza ad ogni round un canale diverso per comunicare con il proprio avversario, sta a lui calcolare dinamicamente se poi dovrà avanzare al round successivo o meno, in base alla propria vittoria o sconfitta.

2.2.2 Sincronizzazione tra Giocatori

Per risolvere il problema di deadlock, ogni attore calcola il proprio ruolo per il round corrente. Il loro ruolo viene determinato dalla posizione del giocatore nell'albero del torneo; I giocatori a sinistra assumono il ruolo di **Leader** (**Server**), e quelli a destra assumono il ruolo di **Follower** (**Client**):

$$\text{isLeader} = \left(\left(\frac{\text{ID}}{2^r} \right) \bmod 2 == 0 \right) \quad (1)$$

Questo esclude la possibilità di deadlock, poiché in ogni battaglia ci sarà sempre un giocatore che attende e uno che invia.

2.2.3 Svolgimento della battaglia

Durante ogni battaglia, il **Follower** genera un numero casuale tra 0 e 20. Il **Leader** riceve questa scelta e genera a sua volta un numero casuale. La somma dei due numeri determina il vincitore. Il giocatore a sinistra (*Leader*) vince se la somma è pari, mentre il giocatore a destra (*Follower*) vince se la somma è dispari. Il vincitore avanza al round successivo, mentre lo sconfitto termina il proprio flusso.

2.3 Conclusioni e Output

Il design decentralizzato ha semplificato la gestione dei canali, eliminando la necessità di un coordinatore centrale e permettendo a ogni giocatore di gestire autonomamente le proprie comunicazioni.

Un approccio centralizzato avrebbe permesso un'output più ordinato, ma avrebbe introdotto complessità aggiuntiva nella gestione dei canali e nella sincronizzazione, oltre a rappresentare un singolo punto di fallimento.